Klein 4B on an Apple M5: running a 16 GB image model in 3.5 GB

Project report · FLUX.2 [klein] 4B, text-to-image, 1024×1024 · measured on a MacBook Air M5 (16 GB) · October 2026

Executive summary

Same model, same math, pixel-equivalent (Tier E; weight streaming itself proven bit-identical): peak RAM 10.1 → 3.5 GB, for ~2–3% extra time. Separately established: Klein's compute is already at ~95% of this chip's ceiling, so we stopped chasing speed with evidence, not fatigue.

	Baseline	Final exact pipeline
Peak memory (macOS)	**10.08 GB** (exp-011, median of 3; range 9.74–10.24)	**3.50 GB** (exp-061, both rounds); p2/p3 runs: 3.53 GB
End-to-end time per image	**18.92 s**: cold single image, stock decode, whole-model compile	**20.67–21.59 s**: warm, throttled steady state, exact chunked decode. The same pipeline with everything resident measured 21.51–21.85 s, so streaming adds no measurable end-to-end time
Cost inside the denoise loop	–	+2.1–2.8% per step (exp-054, two rounds)
Quality vs baseline image	–	Streaming is **bit-identical** to the same-mode resident run (14/14 runs). End-to-end vs the baseline image: **Tier E**, p1: 46.9 dB · p2: 39.0 dB **(worst)**; worst max pixel diff 165/255, from float rounding in the chunked decode and per-block compile
Swap	flat	flat in every final-pipeline run

Why the two times are measured differently. The baseline is a cold single image on a cool chip. The final numbers are steady-state runs on a fanless machine that throttles its GPU about 14% under sustained load, and they include the exact low-memory decoder (0.9–1.2 s slower than the stock decoder). The like-for-like comparison is the resident run of the same pipeline, and against that the streaming changes cost nothing measurable end to end.

What is not claimed yet. We have not run the pipeline on a real 8 GB Mac. 3.5 GB leaves ample room on paper, but MLX's memory-limit setting turned out to be a soft limit, so no simulated '8 GB' claim is valid. The real-hardware test is pending.

Speed. The 1.3× speed goal was closed with evidence. The model's matrix multiplies already run at 95% of the measured peak of the M5's neural accelerators, and every remaining lever has a measured ceiling between 0.89× and 1.11×.

1. The problem in plain terms

In plain terms. An image model like Klein works in three stages, a bit like a small print shop. A **prompt reader** turns your sentence into numbers. A **factory with 25 workstations** then refines a noisy canvas, passing it down the line four times. Finally a **pixel printer** turns the refined canvas into a 1024×1024 picture. Each stage has its own tools, the 'weights', and by default the software keeps every tool for every stage on the workbench at once, plus a lot of half-finished material. On a 16 GB laptop that does not fit: the computer starts swapping to disk and slows down.

Precisely. FLUX.2 [klein] 4B in bf16 has a 7.5 GB Qwen3 text encoder (36 layers, of which 27 are used), a 7.2 GB transformer (5 double-stream + 20 single-stream blocks) and a 0.16 GB VAE. The stock 8-bit mflux setup peaks at 14.12 GB and swaps (exp-002). A fully resident bf16 stack exceeds 16 GB and was never runnable.

The project had two goals of equal weight:

Speed: at least 1.3× faster end to end with equivalent output. **Outcome: closed.** The model is already efficient on this chip; section 2 shows the evidence.
Memory: peak memory at or below 8 GB (stretch: 6 GB) without worse quality. **Outcome: 3.50 GB with pixel-equivalent output**, pending confirmation on real 8 GB hardware.

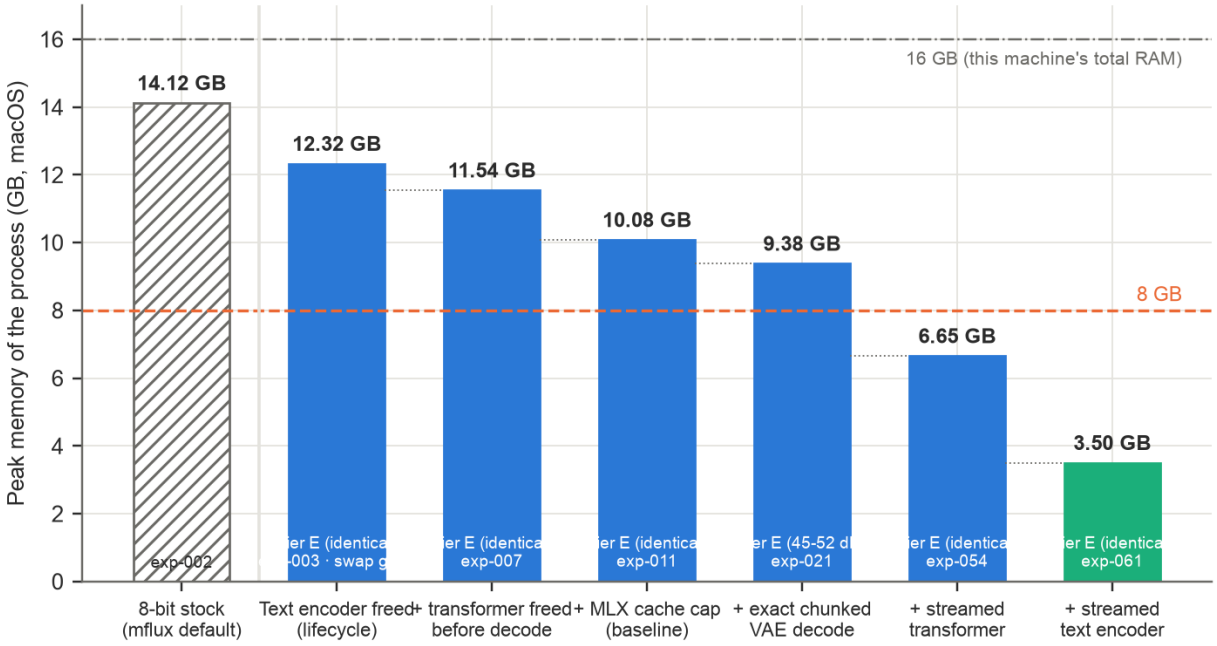


Figure 1. The memory waterfall. *Peak process memory (macOS footprint) as each exact technique is added, bf16 throughout. The hatched bar is the stock 8-bit mflux configuration (exp-002), shown for reference. A fully resident bf16 pipeline exceeds the machine's 16 GB and was never runnable, so the bf16 chain starts with the text encoder freed after use. The first bf16 step still swapped. Every drop is Tier E; the dB range is the chunked decoder's PSNR on the three test prompts.*

2. The speed investigation (closed)

In plain terms. Before trying to make the factory faster, we timed every workstation and asked how close each one already is to the chip's physical limit. The answer: very close. The heavy lifting is matrix multiplication, and on the M5 it runs on dedicated neural-accelerator hardware that is already busy almost all the time. You cannot speed up a machine that is already running near full throttle by rearranging the paperwork around it.

Precisely. One denoise step takes 3.87 s (compiled, cool chip; exp-030). Microbenchmarks at the exact shapes (exp-031) put the matrix multiplies at 2.67 s and attention at 0.75 s per step, leaving 0.45 s (12%) for everything else. Time-weighted, the matmuls run at 10.60 TFLOPS: 95% of the measured bf16 peak (11.15) and 92% of the best rate measured in any precision (11.52). Attention runs at 8.7 TFLOPS. Forcing a pre-M5 GPU architecture drops peak matmul to 3.0 TFLOPS: the baseline already gets a 3.7× boost from the neural accelerators.



Figure 2. Where one denoise step goes *(exp-030 compiled step, split with exp-031 microbenchmarks). Compute-bound work near peak is 88% of the step.*

The closure chain

Roofline. 95% of peak: the matmuls cannot get meaningfully faster.
Exact fusion (merging the small operations between the matmuls). Even if all of the 12% remainder vanished, the image would be ≤ 1.11× faster; the three concrete candidates are estimated at ≤ 1.07×.
Reusing block outputs between steps (caching). 28.4% of block-steps changed little (cosine > 0.98), but only the first four blocks are stable, about 14% of the compute: ceiling ≤ 1.09× (exp-040).
Merging duplicate tokens. A raw 53.9% of image patches looked like near-copies, but after subtracting the average patch only 0.016% remain: ceiling ≈ 1.00× (exp-041).
8-bit weights. Measured 20.41 s vs 18.20 s for bf16 denoise: 0.89×, i.e. slower (exp-015).

In plain terms. The token-merging result needs a word. Picture 4,096 arrows that all lean in roughly the same direction because of a shared background component. Measured naively, every arrow looks like a near-copy of its neighbours. Remove the shared lean and almost none of them are actually alike. The 'duplication' was a property of the whole crowd, not of individual pairs, so there was nothing to merge.

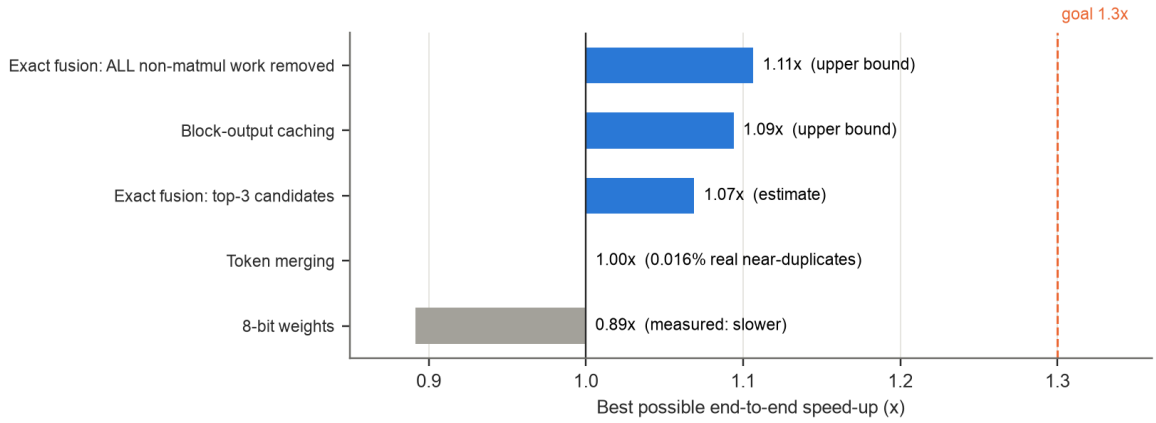


Figure 3. Speed ceilings of every lever we examined, *as the best possible end-to-end speed-up. None reaches the 1.3× goal (dashed). The fusion and caching bars are upper bounds; the 8-bit bar is a measurement.*

We count this as a positive finding: the model is already efficient on this hardware, and we proved it rather than assumed it. Future speed work belongs on workloads with a different profile: reference-image editing (longer sequences) or memory-bandwidth-bound models such as LLM decoding, where the deferred fusion pattern is

expected to pay.

3. The memory result

Each stage was taken apart separately. Every technique below keeps the mathematics of the model unchanged. Where results are bit-identical we say so. Where float rounding changes the last bits we give the PSNR (Tier E means above 40 dB, which is indistinguishable by eye).

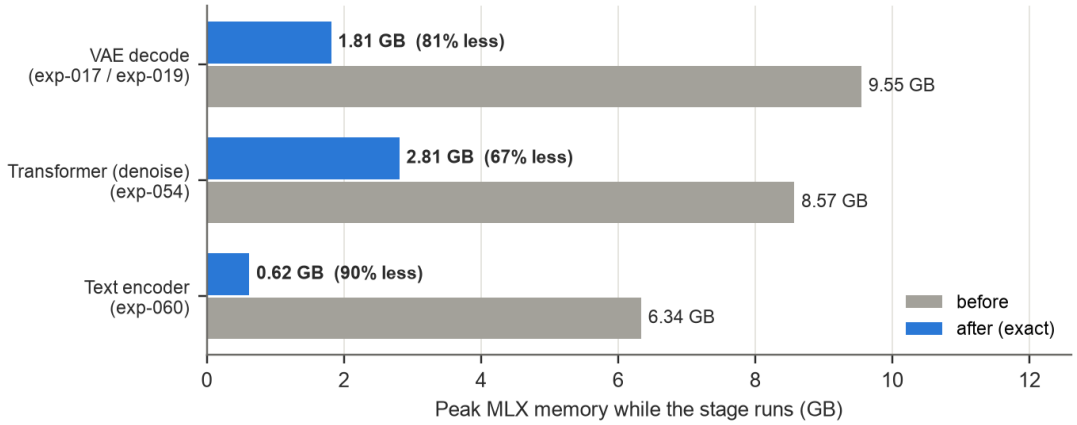


Figure 4. Per-stage memory before and after *(MLX peak while the stage runs, same execution mode within each pair).*

3.1 The pixel printer (VAE decoder)

In plain terms. The printer seemed to need 9.5 GB, more than the 25-workstation factory. Timing it one step at a time showed that no single step needed more than 4.3 GB. The rest was paperwork piling up: the software plans the whole job before doing any of it, and keeps every intermediate sheet until the very end. The fix was to print the picture in horizontal strips, carrying a thin overlap between strips. One subtlety: the colour-balancing step (GroupNorm) must see the whole page, so we measure the statistics over the full page first and only then print strip by strip. That is what makes it exact; the stock tiled mode skips this, which leaves a visible seam.

Precisely. Stock decode peaks at 9.55 GB; the largest single module, profiled eagerly, peaks at 4.33 GB with bit-identical output (exp-014), so the excess is lazy-graph lifetime. Evaluation barriers between blocks: 7.26 GB, bit-identical (exp-018). Exact row-chunked conv / upsample-conv with global-statistic GroupNorm: **1.81 GB**, PSNR p1 52.3 dB, p2 45.2 dB, p3 46.1 dB (exp-019), about 0.8 s slower.

3.2 The 25-workstation factory (transformer)

In plain terms. Instead of keeping all 25 workstations' tools on the floor, we keep only the station that is working and bring in the next one's tools while it works, then clear the previous station. The obvious worry was that fetching tools from the disk would make the line wait. It doesn't: fetching one station's tools takes about 24% of the time that station spends working, so the delivery is always early. The real cost turned out to be different. Forcing the line to stop and check after every station left small gaps. Letting one station's work stay queued while the next begins (a 'lag of one') closed most of them.

Precisely. Each block's weights live in their own file (0.274 GB on average). Cold SSD read speed after a page-cache purge: 6.22–6.46 GB/s, so about 44 ms to load a block vs about 182 ms to compute it. Prefetching the next block hides **99.9%** of the cold load stall (exp-052). Eager streaming still cost +10.1% per step, and +6.6% of that came from the per-block synchronisation alone. With each block compiled separately and a lag-1 barrier, the cost is **+2.1–2.8% per step** and the denoise peak is **2.81 GB** instead of 8.57 GB (exp-054). The output is bit-identical to the resident run of the same mode in all 14 streaming runs.

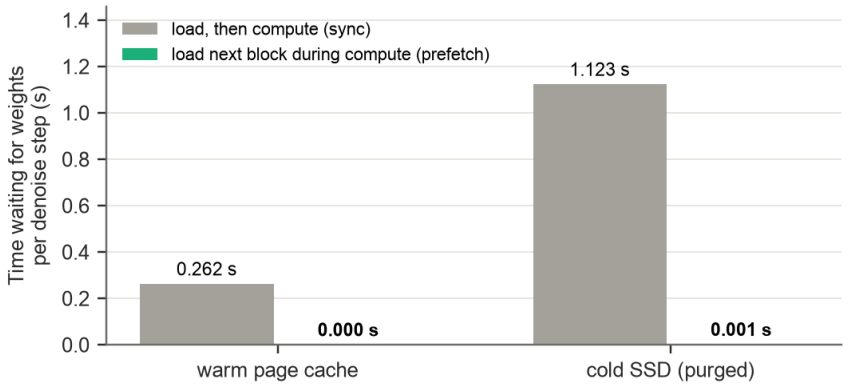


Figure 5. Waiting time per denoise step. *Loading the next block while the current one computes hides 99.9% of the cold-SSD stall (exp-051 warm, exp-052 cold; medians of two rounds).*

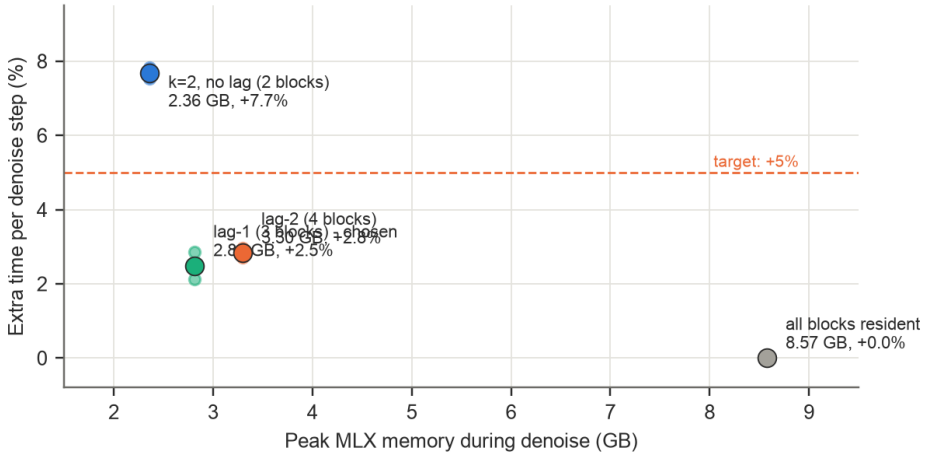


Figure 6. The residency knob. *More blocks in flight cost memory and buy back the synchronisation gaps. Lag-1 meets the +5% target; lag-2 buys nothing more (exp-053, exp-054; small dots are individual rounds).*

3.3 The prompt reader (text encoder)

In plain terms. The prompt reader is a 36-layer language model, but FLUX.2 only reads its notes from layers 9, 18 and 27. The last nine layers do work nobody uses. We also noticed it was loading a 0.7 GB dictionary of 151,936 words to look up at most 512 of them. So we now read only the 27 useful layers, one at a time, and look up only the words in the prompt.

Precisely. Stock peak 6.34 GB, of which the used weights are 5.80 GB; the lazy-graph share is small. Per-layer evaluation barriers made it **worse**, 7.76 GB, because forcing evaluation also runs the nine dead layers that the lazy graph had skipped. Streaming the 27 used layers plus a row-gathered embedding: **0.62 GB**, bit-identical, and the cold encode got faster (0.83 s vs 0.90 s; exp-060).

3.4 The composed pipeline

Precisely. All three together (exp-061, cold SSD, two rounds): **3.50 GB** peak process memory, swap flat, image bit-identical to the same pipeline with a resident text encoder. The peak is now the denoise stage (2.81 GB of MLX arrays plus the Python and MLX runtime). Stage peaks: encode 0.89 GB, load_transformer 0.55 GB, decode 2.51 GB.

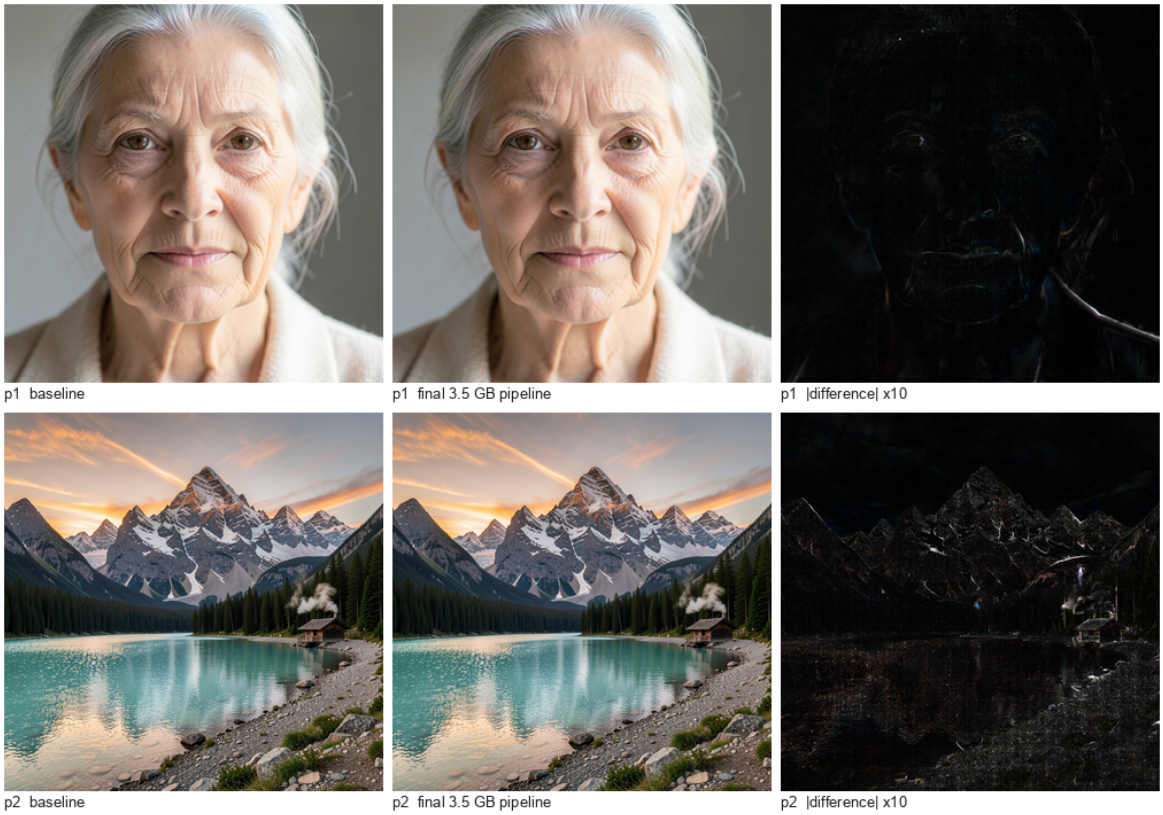


Figure 7. Baseline vs the final 3.5 GB pipeline, *prompts p1, p2. PSNR vs baseline: p1 46.9 dB, p2 39.0 dB (worst: p2). The difference images are amplified ten times; they show float-rounding noise, not structure.*

4. How we kept it honest

A memory or speed claim is only as good as the measurement behind it. These are the rules we followed and the traps they caught.

A fanless laptop gets slower as it warms up

In plain terms. The test machine has no fan. After a few minutes of work it gets warm and quietly slows down, so the fifth run of anything is slower than the first. Comparing 'old method first, new method second' would therefore flatter whichever ran first.

Precisely. Five back-to-back runs drifted from 23.5 s to 27.5 s (exp-001) and 25.5 s to 31.9 s (exp-002). Sustained runs hold the GPU at ~1179 MHz vs ~1365 MHz cold, while macOS still reports thermal pressure 'Nominal'. Every comparison therefore alternates A and B in separate processes with a 150 s cool-down gated on thermal state, and powermetrics logs the GPU clock for every run.

Warm vs cold storage

In plain terms. The first streaming test looked too good: weights 'loaded' at 26 GB/s, faster than the SSD can read. The 16 GB machine was silently keeping the files in spare memory. An 8 GB Mac could not do that, so we emptied that cache before every step to force real disk reads.

Precisely. Warm page-cache reads: 17.4–17.8 GB/s. After `purge`: 6.22–6.46 GB/s. All headline streaming numbers are cold.

Predict first, then measure

Precisely. Before the cold test, this prediction was written down: a block is ~0.29 GB; at ≥ 2 GB/s it loads in < 150 ms vs ~180 ms of compute, so prefetch should hide cold reads with margin. Measured: 0.274 GB, 6.2 GB/s, 44 ms vs 182 ms, 99.9% hidden. **Confirmed.** Writing the expectation down first keeps a lucky number from being read as a designed one.

Compare like with like (rule R1)

In plain terms. The same model run two slightly different ways (all at once, or step by step) can produce images that differ in the last decimal places. If we compared a step-by-step candidate against an all-at-once reference, we could blame our change for a difference it did not cause.

Precisely. Eager vs compiled latents after four steps: cosine p1 0.9996, p2 0.9994, p3 0.9967 (exp-040). Every bit-identity claim is made against a reference produced in the same execution mode.

Swapping runs don't count (R3) and simulations aren't evidence (R2)

Precisely. Any run whose swap usage grew is marked 'memory-only', and its time is never quoted. MLX's memory limit is soft: capped at 8 GB, a run still peaked at 8.54 GB; capped at 6 GB, at 8.06 GB (exp-022). So no '8 GB machine' claim is made from a simulation.

Two quality tiers

In plain terms. Some changes only reorder arithmetic, so the picture is the same to within rounding: **Tier E**, checked by PSNR above 40 dB or bit-identity. Others genuinely approximate, like smaller number formats: **Tier A**, checked with a perceptual metric (LPIPS below 0.05) and side-by-side images. Nothing in the headline is Tier A.

5. What didn't work

Attempt	Number that ended it
8-bit weights for speed	denoise 20.41 s vs 18.20 s bf16: 12% slower (compute-bound; unpacking adds work)
8-bit weights for memory (Tier A)	LPIPS vs bf16 0.027 / 0.044 / 0.081: fails 0.05 on p3
mflux tiled VAE	LPIPS 0.058 / 0.090 / 0.110 vs bf16 and a visible seam (per-tile GroupNorm)
Per-layer barriers on the text encoder	7.76 GB vs 6.34 GB stock: forced the dead layers to run
mx.set_memory_limit as an 8 GB simulation	soft limit: 8.54 GB at an 8 GB cap, 8.06 GB at 6 GB
Block-output caching	only ~14% of compute is static: ≤ 1.09×
Token merging	0.016% near-duplicates after de-meaning
Exact kernel fusion (deferred)	remainder is 12% of a step: ≤ 1.11×
Lag-2 streaming	+2.8% vs +2.5% for lag-1, with 0.5 GB more resident

6. Reusable patterns

The point of the project is an optimizer whose lessons carry to other models. Each entry below is written up in library/patterns/ with no reference to Klein in its matching rule.

Pattern	Applies beyond Klein to	Measured here	Tier
Exact row-chunked VAE decode (global GroupNorm stats)	every convolutional VAE: SD, SDXL, Flux, video VAEs	9.55 → 1.81 GB	E
Eval barriers between blocks	any lazily evaluated graph (MLX, JAX)	VAE 9.55 → 7.26 GB; never before truncating dead layers	E, identical
Block weight streaming + prefetch + lag-1	DiTs, LLM layers, any sequential block stack	8.57 → 2.81 GB, +2.1–2.8%/step	E, identical
Dead-layer truncation + row-gathered embeddings	any encoder used for intermediate hidden states; any large vocabulary	6.34 → 0.62 GB	E, identical
Allocator cache cap	any MLX workload; PyTorch caching allocator analogue	macOS footprint –1.28 GB, no speed cost	E, identical
Lifecycle: free each stage after last use	any multi-model pipeline	15.99 → 9.51 GB MLX (8-bit stack)	E, identical
Per-layer quantization sensitivity scan	any model (tool, not rewrite)	109 layers ranked; floor verified 0	tool

The residency knob (transformer denoise, cold SSD):

Setting	Blocks in memory	Denoise peak (MLX)	Extra time per step
k=2, no lag	2	2.36 GB	+7.5–7.8%
lag-1 (chosen)	3	2.81 GB	+2.1–2.8%
lag-2	4	3.30 GB	+2.7–3.0%
all resident	25	8.57 GB	0 (reference)

Lesson: streaming cost is bubbles, not loading. With prefetch, waiting for weights fell to 1.4 ms per step, yet streaming still cost +10.1% (eager). Synchronisation gaps explained +6.6%, and the remaining 2.4–3.5% is memory-bus contention between the weight transfer and the GPU, now recorded in the hardware profile as a generic cost of streaming on unified memory.

7. Open items

• **Real 8 GB Mac test.** The only valid confirmation of the memory result. The candidate local machine (M1, 8 GB) needs about 20 GB of free disk first; an M1 also lacks the M5's neural accelerators, so it would confirm memory, not speed.
• **Generalization (Phase 4).** Run the frozen patterns on Z-Image Turbo and a small LLM decoder, and record which patterns fire.
• **Upstreaming.** The exact chunked VAE decode, text-encoder dead-layer truncation and block streaming are candidates for mflux.
• **Speed on other workloads.** Reference-image edits and bandwidth-bound LLM decoding, where the deferred fusion pattern applies.

Appendix A. Every experiment

Speed-up is against the 18.92 s baseline (exp-011, median of 3) and is shown only for end-to-end runs with flat swap. 'memory-only' means swap grew, so the time is invalid (R3). PSNR is against the baseline images; 'same-mode' is the R1 bit-identity check. Profiling, microbenchmark and stage-only runs have no end-to-end time.

Exp	Config	Cache cap GB	Quality ref	Total s	Speed-up	OS peak GB	PSNR dB / same-mode	Swap
exp-000-smoke	q8 free_te	-	-	24.85	memory-only	12.40	-	grew
exp-001	q8 free_te	-	-	25.42	memory-only	12.65	-	grew
exp-002	q8 stock	-	q8_free_te	26.61	memory-only	14.12	p1:= p2:= p3:=	grew
exp-003	bf16 free_te	-	q8_free_te	26.67	memory-only	12.32	p1:29.4 p2:27.5 p3:20.7	grew
exp-004	q8 free_te tiledVAE	-	q8_free_te	39.70	memory-only	11.90	p1:26.5 p2:26.3 p3:27.1	grew
exp-005	q8 free_te	-	q8_free_te	21.65	memory-only	11.92	p1:=	grew
exp-006	q8 staged	-	q8_free_te	20.88	memory-only	11.70	p1:=	grew
exp-007	bf16 staged	-	bf16_free_te	19.50	0.97x	11.54	p1:=	flat
exp-008	q8 staged tiledVAE	-	q8_free_te	21.10	0.90x	8.71	p1:26.5	flat
exp-009	bf16 staged	0.5	bf16_free_te	19.57	0.97x	10.26	p1:=	flat
exp-010	q8 staged tiledVAE	0.5	q8_free_te	20.66	0.92x	6.04	p1:26.5	flat
exp-010-rep-1	q8 staged tiledVAE	0.5	q8_free_te	20.31	0.93x	6.04	p1:26.5	flat
exp-010-rep-2	q8 staged tiledVAE	0.5	q8_free_te	20.42	0.93x	6.04	p1:26.5	flat
exp-010-rep-3	q8 staged tiledVAE	0.5	q8_free_te	20.48	0.92x	6.05	p1:26.5	flat
exp-011-bf16-1	bf16 staged	0.5	bf16_free_te	19.20	0.99x	10.08	p1:=	flat
exp-011-bf16-2	bf16 staged	0.5	bf16_free_te	18.86	1.00x	10.24	p1:=	flat
exp-011-bf16-3	bf16 staged	0.5	bf16_free_te	18.92	1.00x	9.74	p1:=	flat
exp-011-q8-1	q8 staged	0.5	q8_free_te	20.02	0.95x	9.80	p1:=	flat
exp-011-q8-2	q8 staged	0.5	q8_free_te	20.00	0.95x	9.80	p1:=	flat
exp-011-q8-3	q8 staged	0.5	q8_free_te	20.04	0.94x	9.80	p1:=	flat
exp-012-smoke	bf16 staged	0.5	bf16_free_te	-	memory-only	10.82	p1:=	grew
exp-013	q8 free_te tiledVAE	0.5	q8_free_te	22.81	0.83x	6.16	p1:26.5 p2:26.3 p3:27.1	flat
exp-014	bf16	-	-	-	-	-	p1:=	-
exp-015-bf16-1	bf16 staged	0.5	bf16_free_te	-	-	10.87	p1:=	flat
exp-015-bf16-2	bf16 staged	0.5	bf16_free_te	-	-	10.88	p1:=	flat
exp-015-bf16-3	bf16 staged	0.5	bf16_free_te	-	-	10.87	p1:=	flat
exp-015-q8-1	q8 staged	0.5	q8_free_te	-	-	10.94	p1:=	flat
exp-015-q8-2	q8 staged	0.5	q8_free_te	-	-	10.89	p1:=	flat
exp-015-q8-3	q8 staged	0.5	q8_free_te	-	-	10.91	p1:=	flat
exp-016-lat-p2	bf16 staged	0.5	bf16_free_te	-	memory-only	10.52	p2:=	grew
exp-016-lat-p3	bf16 staged	0.5	bf16_free_te	-	memory-only	10.32	p3:=	grew
exp-017-stock-p1	bf16	0.5	bf16_free_te	-	-	10.66	p1:=	flat
exp-017-stock-p2	bf16	0.5	bf16_free_te	-	memory-only	10.29	p2:=	grew
exp-017-stock-p3	bf16	0.5	bf16_free_te	-	-	10.29	p3:=	flat
exp-018-barriers-p1	bf16 vae:barriers	0.5	bf16_free_te	-	-	7.28	p1:=	flat
exp-018-barriers-p2	bf16 vae:barriers	0.5	bf16_free_te	-	-	7.28	p2:=	flat
exp-018-barriers-p3	bf16 vae:barriers	0.5	bf16_free_te	-	-	7.28	p3:=	flat
exp-019-chunk128-p1	bf16 vae:chunked	0.5	bf16_free_te	-	-	2.42	p1:52.3	flat
exp-019-chunk128-p2	bf16 vae:chunked	0.5	bf16_free_te	-	-	2.40	p2:45.2	flat
exp-019-chunk128-p3	bf16 vae:chunked	0.5	bf16_free_te	-	-	2.46	p3:46.1	flat
exp-020-chunk64-p1	bf16 vae:chunked	0.5	bf16_free_te	-	-	2.41	p1:52.1	flat
exp-020-chunk256-p1	bf16 vae:chunked	0.5	bf16_free_te	-	-	2.77	p1:52.6	flat
exp-021-chunk-1	bf16 staged vae:chunked	0.5	bf16_free_te	20.15	memory-only	9.38	p1:52.3	grew
exp-021-chunk-2	bf16 staged vae:chunked	0.5	bf16_free_te	20.26	0.93x	9.18	p1:52.3	flat
exp-021-chunk-3	bf16 staged vae:chunked	0.5	bf16_free_te	19.72	0.96x	9.38	p1:52.3	flat
exp-021-stock-1	bf16 staged	0.5	bf16_free_te	19.10	memory-only	10.19	p1:=	grew
exp-021-stock-2	bf16 staged	0.5	bf16_free_te	19.35	0.98x	9.74	p1:=	flat
exp-021-stock-3	bf16 staged	0.5	bf16_free_te	19.07	0.99x	10.28	p1:=	flat
exp-022-cap6	bf16 staged vae:chunked	0.5	bf16_free_te	23.59	0.80x	8.06	p1:52.3	flat
exp-022-cap8	bf16 staged vae:chunked	0.5	bf16_free_te	20.37	0.93x	8.54	p1:52.3	flat
exp-030	bf16	-	-	-	-	-	-	-
exp-031-arch-default	microbench	-	-	-	-	-	-	-

Exp	Config	Cache cap GB	Quality ref	Total s	Speed-up	OS peak GB	PSNR dB / same-mode	Swap
exp-031-arch-g16g	microbench	-	-	-	-	-	-	-
exp-032	bf16	-	-	-	-	-	-	-
exp-040	bf16	-	-	-	-	-	-	-
exp-041	bf16	-	-	-	-	-	-	-
exp-050	disk	-	-	-	-	-	-	-
exp-051-prefetch-1	bf16 prefetch vae:chunked	0.5	-	-	-	6.65	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-051-prefetch-2	bf16 prefetch vae:chunked	0.5	-	-	-	6.65	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-051-resident-1	bf16 resident vae:chunked	0.5	-	-	memory-only	9.53	baselines/bf16_free_te:46.9 | same-mode:	grew
exp-051-resident-2	bf16 resident vae:chunked	0.5	-	-	-	9.59	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-051-sync-1	bf16 sync vae:chunked	0.5	-	-	-	6.65	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-051-sync-2	bf16 sync vae:chunked	0.5	-	-	-	6.65	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-052-barrier-1	bf16 barrier vae:chunked	0.5	-	-	-	9.53	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-052-barrier-2	bf16 barrier vae:chunked	0.5	-	-	-	9.56	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-052-disk	disk	-	-	-	-	-	-	-
exp-052-prefetch-cold-1	bf16 prefetch-cold vae:chunked	0.5	-	-	-	6.65	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-052-prefetch-cold-2	bf16 prefetch-cold vae:chunked	0.5	-	-	-	6.65	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-052-sync-cold-1	bf16 sync-cold vae:chunked	0.5	-	-	-	6.65	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-052-sync-cold-2	bf16 sync-cold vae:chunked	0.5	-	-	-	6.65	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-053-pblockc-1	bf16 prefetch_blockc-cold vae:chun	0.5	-	22.71	0.83x	6.65	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-053-pblockc-2	bf16 prefetch_blockc-cold vae:chun	0.5	-	22.84	0.83x	6.65	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-053-rblockc-1	bf16 resident_blockc vae:chunked	0.5	-	21.67	0.87x	9.53	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-053-rblockc-2	bf16 resident_blockc vae:chunked	0.5	-	21.80	0.87x	9.56	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-053-rcomp-1	bf16 resident_compiled vae:chunked	0.5	-	21.41	0.88x	9.56	baselines/bf16_free_te:52.1	flat
exp-053-rcomp-2	bf16 resident_compiled vae:chunked	0.5	-	21.77	0.87x	9.56	baselines/bf16_free_te:52.1	flat
exp-054-lag1-1	bf16 prefetch_blockc-cold-lag1 vae	0.5	-	22.10	0.86x	6.65	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-054-lag1-2	bf16 prefetch_blockc-cold-lag1 vae	0.5	-	21.80	0.87x	6.65	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-054-lag2-1	bf16 prefetch_blockc-cold-lag2 vae	0.5	-	21.85	0.87x	6.65	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-054-lag2-2	bf16 prefetch_blockc-cold-lag2 vae	0.5	-	22.21	0.85x	6.69	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-054-rblockc-1	bf16 resident_blockc vae:chunked	0.5	-	21.51	0.88x	9.53	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-054-rblockc-2	bf16 resident_blockc vae:chunked	0.5	-	21.85	0.87x	9.53	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-060-barrier-1	bf16 barrier	0.5	-	-	-	8.07	-	-
exp-060-barrier-2	bf16 barrier	0.5	-	-	-	8.07	-	-
exp-060-stock-1	bf16 stock	0.5	-	-	-	6.82	-	-
exp-060-stock-2	bf16 stock	0.5	-	-	-	6.81	-	-
exp-060-stream-1	bf16 stream	0.5	-	-	-	0.93	-	-
exp-060-stream-2	bf16 stream	0.5	-	-	-	0.93	-	-
exp-060-truncate-1	bf16 truncate	0.5	-	-	-	6.81	-	-
exp-060-truncate-2	bf16 truncate	0.5	-	-	-	6.81	-	-
exp-061-full-1	bf16 prefetch_blockc-cold-lag1 vae	0.5	-	20.67	0.92x	3.50	baselines/bf16_free_te:46.9 | same-mode:	flat
exp-061-full-2	bf16 prefetch_blockc-cold-lag1 vae	0.5	-	21.59	0.88x	3.50	baselines/bf16_free_te:46.9 | same-mode:	flat

Exp	Config	Cache cap GB	Quality ref	Total s	Speed-up	OS peak GB	PSNR dB / same-mode	Swap
exp-062-full-p2	bf16 prefetch_blockc-cold-lag1 vae	0.5	-	21.27	0.89x	3.53	baselines/bf16_free_te:39.0	flat

Appendix B. Machine, software, protocol

Machine	MacBook Air Mac17,3 (fanless), Apple M5: 4 performance + 6 efficiency CPU cores, 8-core GPU with neural accelerators, Metal 4, 16 GB unified memory, macOS 26.6
Measured device facts	bf16 matmul peak 11.15 TFLOPS (3.04 without neural accelerators); memory bandwidth 135 GB/s (read+write); SSD 6.22–6.46 GB/s cold
Software	Python 3.12.15, MLX 0.32.4, mflux 0.23.0, numpy 2.5.4, LPIPS (AlexNet) via torch 2.14.1
Model	black-forest-labs/FLUX.2-klein-4B @ e7b7dc2, Apache-2.0; bf16; 1024×1024, 4 steps, guidance 1.0, seed 42; prompts p1 portrait, p2 landscape, p3 text sign
R1 execution mode	every correctness comparison uses a reference produced in the same execution mode (eager vs compiled)
R2 simulation	mx.set_memory_limit is a soft limit; no smaller-machine claim is made from it
R3 timing validity	swap-grown runs are memory-only; timing comparisons alternate A/B processes with cool-downs
Tiers	E: PSNR > 40 dB or bit-identical. A: LPIPS < 0.05 plus visual review